BlockFund

PROJECT DOCUMENTATION, VERSION 2, OCTOBER 2026

BlockFund community token

A student-led experiment in blockchain technology: an ERC-20 token, our own exchange and faucet contract, and the website that ties them together.

COORDINATOR
George Dedempilis
github.com/georgeded
LinkedIn

COORDINATOR
Dean Kok
github.com/deank055
LinkedIn

PROJECT
Amsterdam, Netherlands
blockfund.onrender.com
Source code

BlockFund is built purely for education and experimentation. It runs on the Base Sepolia testnet, so every token and transaction is simulated and has no monetary value. It is not an investment product and must never be bought with real funds.

Contents

01 Abstract — Main idea

02 Project description — What BlockFund is

03 Architecture — Stack and file structure

04 Smart contracts — Token, swap, faucet

05 Features — What the website offers

06 User guide — From wallet to holder

07 Running and deploying — For developers

08 Reference — Network, rates, API

09 Challenges and solutions — What we learned

10 Resources and references — Credits

01 Abstract

BlockFund is a student-led educational project that gives hands-on experience with blockchain technology and decentralized finance (DeFi) in a risk-free setting. Everything runs on **Base Sepolia**, the public testnet of Base, an Ethereum layer-2 network, so users can experiment without any financial risk.

The project has three parts: the **BlockFund (BF)** ERC-20 token, **BlockFund Swap** (our own exchange contract with a built-in daily faucet), and a website that explains the project, lets anyone acquire BF, and unlocks holder-only features: crypto news, live market graphs and a group chat. The aims are to build technical skills, understand how blockchains work end to end, and learn in the open.

02 Project description

BlockFund was created by two students as a portfolio project. The first version (2024) ran on the Ethereum Sepolia testnet and used a third-party Uniswap testnet deployment for trading. In 2026 the project was rebuilt:

- **New network.** Ethereum Sepolia faucets started requiring a mainnet balance and the network reached its planned end-of-life, so newcomers could no longer get free test ETH. BlockFund moved to Base Sepolia, where several faucets still hand out test ETH for free.
- **Our own exchange.** Instead of relying on an external app, BlockFund now ships its own swap contract. Users pay with ETH, WETH or test USDC and receive BF at a fixed rate, can sell BF back, or claim a small free amount of BF once a day.
- **Wallet sign-in.** Holders log in by signing a message with their wallet, and holder status is read directly from the blockchain instead of a third-party explorer API.
- **Redesigned website.** A minimal, futuristic interface inspired by ethereum.org, with light and dark themes and full mobile support.

BF has a fixed supply of 1,000,000 tokens and no monetary value. Holding any amount lets a user log in with their wallet and access the holder area of the site.

03 Architecture

LAYER	TECHNOLOGY
Blockchain	Base Sepolia testnet (chain ID 84532), Solidity 0.8.24
Contracts tooling	solc-js (compile), ganache (in-process test chain), ethers.js v6 (tests and deploy)
Frontend	Plain HTML, CSS and JavaScript; ethers.js v6 on the swap page; Lightweight Charts and TradingView widgets on the graphs page
Backend	Node.js and Express: serves the site, wallet sign-in and sessions, live chat (Server-Sent Events), news and feedback
Database	PostgreSQL (chat messages and feedback)
Data sources	Base Sepolia RPC (balances, swap), Blockscout (BF transfer history), CoinGecko (market data), public RSS feeds (news)
Hosting	Render

Repository structure

```
public/                 Website, served as static files
  index.html            Home page
  pages/                learn, use, swap, participate, news, graphs, chat
  css/style.css         Styles, with light and dark theme tokens
  js/config.js          Network settings and contract addresses (shared with the server)
  js/layout.js          Shared header, login and profile dialogs, footer
  js/site.js            Theme, menu, login and holder sessions (every page)
  js/wallet.js          Wallet discovery, network and token helpers (every page)
  js/*.js               One script per page
  assets/               Logo and icons
  docs/                 This documentation (HTML source and PDF)
server/
  server.js             Express app and routes
  auth.js               Sign-in challenges and signature checks
  chain.js              On-chain BF balance checks
  chat.js               Live chat API (Server-Sent Events)
  feedback.js           Feedback validation, storage and email
  http.js               Async route helper and HTTP errors
  news.js               Merges and caches the RSS news feeds
  session.js            Signed session tokens
  transfers.js          Cached BF transfer history from Blockscout
  store.js              PostgreSQL storage, with JSON files as a local fallback
contracts/
  src/                  BlockFund.sol, BlockFundSwap.sol
  scripts/              compile.js, deploy.js, test.js
```

How the pieces talk

Every page loads `config.js` and `layout.js` first, so the header renders with the page, then `wallet.js` and `site.js` for shared behaviour. Reads from the chain (balances, rates, reserves, faucet timers) go straight to the public Base Sepolia RPC, so they work without a wallet. Anything that changes state (swaps, approvals, faucet claims) is signed in the user's own wallet. The server only reads BF balances to check holders; it never holds private keys or sends transactions.

04 Smart contracts

BlockFund.sol: the BF token

A minimal, standard ERC-20: name `BlockFund`, symbol `BF`, 18 decimals. The whole supply of 1,000,000 BF is minted once to the deployer in the constructor; there is no mint function, so the supply can never grow. It supports `transfer`, `approve` and `transferFrom`, including the common "infinite approval" convention.

BlockFundSwap.sol: exchange and faucet

A fixed-rate exchange desk that holds a reserve of BF. The owner lists payment tokens and sets a rate for each; `address(0)` stands for native ETH.

- **Buying:** `buy(tokenIn, amountIn, minBfOut)`. The user pays ETH (sent with the call) or an approved ERC-20 and receives `amountIn * rate / 10^18` BF from the reserve.
- **Selling:** `sell(tokenOut, bfIn, minOut)`. The user returns approved BF and receives ETH or a listed token, paid from whatever the contract holds.
- **Faucet:** `claim()` sends a fixed amount of BF (25 BF by default) to the caller, at most once per cooldown (24 hours by default) per address.
- **Views for the website:** `supportedTokens()`, `quoteBuy`, `quoteSell`, `reserve`, `nextClaimAt`.
- **Owner controls:** `setRate` (a rate of 0 delists a token), `setFaucet`, `withdraw`, `transferOwnership`.

> **Why fixed rates?** An automated market maker like Uniswap needs real liquidity on both sides and its price moves with every trade. A fixed rate keeps BF predictable and easy to explain, and lets a small reserve serve many users. Rates are expressed as BF per 10^18 base units of the payment token, so tokens with any number of decimals (USDC uses 6) price correctly.

Safety measures

- A reentrancy guard on every function that moves funds.
- Slippage protection: every trade states a minimum output and reverts if it isn't met (the website uses 0.5%).
- Reserve checks before paying out, with clear revert reasons (`BF reserve too low`, `reserve too low`).
- Safe ERC-20 calls that accept tokens which return nothing as well as tokens which return `true`.
- Only real contracts can be listed, BF cannot be priced in BF, and ETH amounts must match exactly.

Tests

`contracts/scripts/test.js` deploys both contracts on an in-process ganache chain and checks: buying with ETH, value and slippage guards, buying with an ERC-20, listing rules, selling BF for ETH, sells limited by the reserve, the faucet and its cooldown, and owner-only controls. Run it with `npm test` in `contracts/`.

05 Features

1. Website

Home, Learn, Use, Swap and Participate pages explain the project and guide new users. The design is minimal and futuristic, uses line-art illustrations that adapt to the theme, and works from large desktop screens down to phones. A sun/moon button switches between **light and dark mode**; the site follows the device setting by default and remembers the user's choice.

2. BlockFund Swap

A swap interface for `BlockFundSwap.sol`. It connects any installed wallet (with a selector when there are several), offers to add and switch to Base Sepolia, lists the payment tokens straight from the contract, shows live quotes, the minimum received and the available liquidity, handles ERC-20 approvals automatically, and links every transaction to the block explorer. A **Sell BF** tab reverses the direction.

3. Daily BF faucet

Next to the swap, users can claim free BF once a day, with a live countdown to their next claim. This lets anyone become a holder for only a tiny gas fee in test ETH.

4. Holder login

Holders log in with **Connect wallet**. The server issues a one-time sign-in message in the EIP-4361 (Sign-In with Ethereum) format, the wallet signs it for free, and the server checks that the signature belongs to that address. Each message expires after five minutes and works only once. The server then reads the address's BF balance on-chain and, if it holds BF, issues a signed session token valid for seven days. On desktop, every installed wallet that supports EIP-6963 (MetaMask, Rabby, Coinbase Wallet, Brave and others) is listed to choose from. On a phone without a wallet in the browser, the dialog offers to open the site inside MetaMask, Coinbase Wallet or Trust Wallet.

There are no passwords. An earlier version let users register any address with a password, which did not prove they owned it; wallet signatures do. Holder pages (news, graphs, chat) show a blurred preview with a login prompt to visitors, and the Holders menu shows a lock until they log in.

5. Crypto news (holders)

Headlines from **CoinDesk, Cointelegraph, Decrypt and The Block**, read from their public RSS feeds. No API key is needed. The server merges the feeds newest-first, caches them for five minutes, and the page refreshes every minute. Articles without an image show the BlockFund logo instead.

6. Live graphs (holders)

- **BlockFund activity:** cumulative BF transfers per day, drawn with TradingView's Lightweight Charts. The server fetches the data from Blockscout and caches it for a minute.
- **Top 10 by 24h volume:** a live table from CoinGecko's free API.
- **Price charts:** an embedded TradingView widget for the largest coins.

7. Group chat (holders)

A real-time chat for holders and coordinators. New messages are pushed to everyone instantly with Server-Sent Events and stored in PostgreSQL, so they survive restarts and redeploys. The sender is taken from the session, so nobody can post as another address. The last 100 messages are shown, and coordinators can clear the chat or restore the last cleared messages.

8. Feedback form

The Participate page has a built-in feedback form (topic, optional name and email, message). The server saves each submission to the database and emails it to the team through Resend's HTTPS API, setting Reply-To to the sender's address. (Render's free plan blocks SMTP, so a plain Gmail setup would not deliver.) A hidden honeypot field and a limit of five messages per hour per visitor keep spam out.

06 User guide

The **Use** page on the website turns most of these steps into a single click.

1. **Install a wallet.** Any EVM browser wallet works, such as MetaMask, Rabby or Coinbase Wallet. Store your recovery phrase safely and never share it; BlockFund will never ask for it.
2. **Add Base Sepolia.** Press *Add network* on the Use page, or add it manually with the values in the Reference section.
3. **Get free test ETH.** Paste your address into a Base Sepolia faucet such as Alchemy, QuickNode or Coinbase CDP (links on the Use page). Circle's faucet also gives test USDC, which the swap accepts.
4. **Get BF.** Open **Swap**, connect your wallet, enter an amount of ETH, WETH or USDC and confirm. For ERC-20 tokens your wallet first asks for an approval. Alternatively, press *Claim* to receive the free daily BF.
5. **Show BF in your wallet.** Press *Add BF to wallet*, or import the token manually with the BF address from the Use page.

6 **Log in.** Press *Log in* in the top bar, then *Connect wallet*, and approve the signature request. If the address holds BF, the lock on the **Holders** menu disappears and News, Graphs and Chat unlock.

> Selling works the same way in reverse: switch to **Sell BF**, pick what you want to receive, approve BF once, and confirm.

07 Running and deploying

Run the website locally

```
npm run build      # installs the server's dependencies
npm start          # http://localhost:8080
```

Without a database, chat and feedback are kept in `server/data/`, which is fine for local development. On Render, the build command is `npm run build` and the start command is `npm start`, with these environment variables:

VARIABLE	PURPOSE
`DATABASE_URL`	PostgreSQL connection string (for example a free Neon database). Required on Render, whose disk is wiped on every deploy.
`SESSION_SECRET`	Long random string that signs sessions, so holders stay logged in across restarts.
`RESEND_API_KEY`	Sends feedback emails through Resend.
`FEEDBACK_TO`	Optional. Feedback recipient, defaults to g.dedebilis@gmail.com.

Test and deploy the contracts

```
cd contracts
npm install
npm test            # compile and run the test suite
read -s -p "Private key: " PRIVATE_KEY && export PRIVATE_KEY && npm run deploy
```

Use a testnet-only key that holds a little Base Sepolia ETH. The deploy script deploys BF, deploys the swap with the default ETH rate and faucet settings, lists WETH and USDC, moves 900,000 BF into the swap's reserve (the deployer keeps 100,000), and prints two addresses. Paste them into `public/js/config.js` as `bfToken` and `swap`. Until then the swap page runs in preview mode and shows quotes only.

08 Reference

Network

Network name	Base Sepolia
Chain ID	84532 (`0x14a34`)
RPC URL	`https://sepolia.base.org`
Currency	ETH (test)
Block explorer	`https://sepolia.basescan.org`

Token and default swap settings

ITEM	VALUE
Token	BlockFund (BF), 18 decimals, 1,000,000 fixed supply
Token contract	`0x89c0452ce2Bd699587Bf3437D9736f4c54966a5D`
Swap contract	`0x92D05af6dCaEEEC25c44eb60332DF96857645774`
ETH	1 ETH = 10,000 BF
WETH	1 WETH = 10,000 BF (`0x4200...0006`)
USDC	1 USDC = 5 BF (`0x036C...CF7e` , Circle test USDC)
Faucet	25 BF per address every 24 hours
Slippage	0.5% minimum-output tolerance in the website

Server API

ENDPOINT	PURPOSE
`POST /api/auth/challenge`	One-time sign-in message for an address
`POST /api/auth/verify`	Check the signed message and BF balance, return a session
`GET /api/session`	Re-check a saved session and the BF balance
`GET /api/news`	Merged RSS news, cached for five minutes
`GET /api/transfers`	BF transfer count and daily history, cached for a minute
`POST /api/feedback`	Send feedback (saved and emailed)
`GET /api/chat`	Chat messages (session required)
`GET /api/chat/stream`	Live chat updates via Server-Sent Events (session required)
`POST /api/chat`	Send a message (session required)
`DELETE /api/chat`	Clear the chat (coordinators)
`POST /api/chat/restore`	Restore the last cleared messages (coordinators)

09 Challenges and solutions

When we started, our experience in front-end design, blockchain integration and backend development was limited. Building visually polished interfaces, reading blockchain data from code, and implementing features like the live chat and the login system all stretched our skills. We closed the gaps through reading, tutorials, and frequent discussions where we brainstormed solutions together.

The 2026 rebuild brought new challenges:

- **A disappearing testnet.** Sepolia faucets started requiring mainnet ETH, which locked out new users. We compared the remaining testnets and faucet requirements and moved to Base Sepolia.
- **Dependence on outside services.** The external swap, the old Etherscan API and the news API all changed or shut down. We replaced them with our own contracts, direct RPC reads, and keyless sources (Blockscout, CoinGecko, RSS).
- **Writing contracts safely.** Moving real value around, even on a testnet, demanded care. We added reentrancy protection, slippage limits and reserve checks, backed by an automated test suite.

- **Maintainability.** Every page repeated hundreds of lines of header and footer code. We introduced a shared layout, a single design system, and a clear `public/`, `server/`, `contracts/` structure.

10 Resources and references

- **Design inspiration:** ethereum.org.
- **Network:** Base Sepolia by Base (Coinbase); public RPC at sepolia.base.org.
- **Blockchain data:** Blockscout (BF transfer history) and BaseScan (explorer links).
- **Contracts tooling:** Solidity, solc-js, ganache, ethers.js.
- **Wallets:** EIP-4361 (Sign-In with Ethereum) messages and EIP-6963 wallet discovery.
- **Email:** Resend.
- **Database:** PostgreSQL.
- **Market data:** CoinGecko API.
- **Charts:** TradingView Lightweight Charts and TradingView widgets.
- **News:** RSS feeds from CoinDesk, Cointelegraph, Decrypt and The Block.
- **Faucets:** Alchemy, QuickNode, Coinbase Developer Platform, Circle.
- **Fonts:** Inter and JetBrains Mono (Google Fonts).